

MIRCHILIME Connect

100 Trucks FASTag Analysis Platform

Everything you need to understand this platform — what it does, how the data flows, what every part is for, how the schedule works, and exactly where to make changes when you come back to this project after months away.

CONTENTS

1. Project Overview	8. WhatsApp Reporting
2. System Architecture	9. Public Tracking Link
3. Files and What They Do	10. How to Make Changes
4. Database Tables	11. Common Problems
5. Schedule — What Runs When	12. Environment Variables
6. API Endpoints	13. Zoho API Reference
7. Dashboard Views	14. Quick Reference

1. Project Overview

This is a **web-based fleet tracking platform** built for MIRCHILIME to monitor trucks in real time using FASTag toll crossing data. It connects to Zoho CRM to fetch active trips, then pings the FASTag API to get each truck's latest toll crossing, showing live positions on a map with full route history.

The platform has two types of trips:

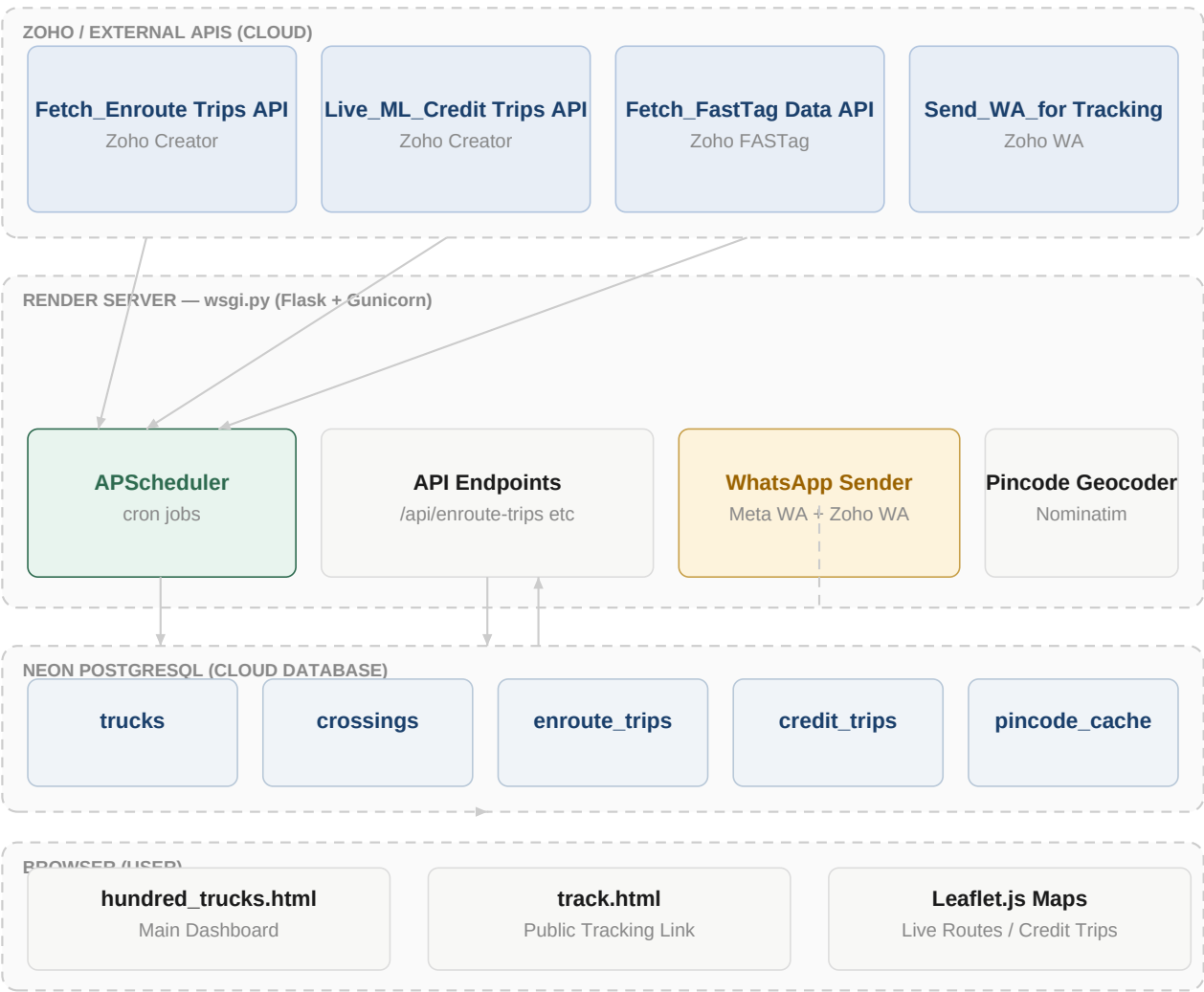
- **Enroute Trips (Live Routes)** — Long-distance inter-state trips. Take 7–20 days. Fetched from Zoho's Fetch_Enroute_Trips API.
- **Credit Trips** — Short-distance local trips within Tamil Nadu. Usually completed in 1–2 days. Fetched from Zoho's Live_ML_Credit_Trips API.

What the platform does:

- Shows all trucks on a single map with their current FASTag position
- Displays the full toll crossing history for each truck in order
- Shows the loading point, pickup time, and the first toll crossed after pickup (real start time)
- Sends individual truck tracking links via WhatsApp automatically at 11:05 AM and 7:05 PM every day
- Auto-syncs trip data from Zoho CRM twice a day
- Provides a public tracking page for each truck — share the link with drivers or customers
- Trip completion detection built for enroute trips — not yet active (Zoho not returning drop_pincode)

2. System Architecture

The system has four layers — External APIs (Zoho), a Backend server on Render, a Cloud Database on Neon, and the Browser frontend. The diagram below shows where each piece lives and how they connect.



Technology Stack

Component	Technology	Purpose
Backend	Python / Flask	Web server and all API logic
Server	Gunicorn (1 worker)	Production WSGI server on Render
Hosting	Render.com	Cloud hosting, auto-deploy from Git

Component	Technology	Purpose
Database	Neon PostgreSQL	Stores all trucks, crossings, trips
Scheduler	APScheduler	Runs sync and ping jobs on a timed schedule
Frontend	HTML + JavaScript	Single-page app, no framework needed
Maps	Leaflet.js	Interactive maps with truck routes and markers
WhatsApp	Meta WA API + Zoho WA API	Send tracking links to phone numbers

3. Files and What They Do

There are only three files you need to know. Everything is in these.

wsgi.py

The entire backend

All API endpoints, database logic, scheduler jobs, Zoho sync, FASTag ping, WhatsApp sending. The most important file. If something breaks on the server, start here.

hundred_trucks.html

The main dashboard

All 6 views (Trucks, Map View, All Routes, Report, Live Routes, Credit Trips) are inside this single HTML file. No external framework — just HTML and JavaScript.

track.html

Public tracking page

Anyone with the link can open this on their phone to see where a specific truck is. Has an expiry date that needs updating periodically.

requirements.txt

Python dependencies

List of Python packages needed to run the server — Flask, psycpg2, APScheduler etc. Run `pip install -r requirements.txt` to set up.

.env

Secret variables (DO NOT commit)

Stores DATABASE_URL, WA_TOKEN, and other secrets. Never push this file to Git. On Render, these are set as Environment Variables in the dashboard.

render.yaml / Render dashboard

Deployment config

Start command: `gunicorn wsgi:application --worker-class sync --workers 1 --preload --timeout 120`. The `--workers 1` is critical — multiple workers cause duplicate WhatsApp messages.

Key variables at the top of wsgi.py

These are the first things to check if an API or integration stops working:

Variable	What it is
NEON_URL	PostgreSQL connection string from Neon dashboard
ZOHO_URL	FASTag data API from Zoho (Fetch_Fast_Tag_Data)
CREDIT_TRIPS_URL	Zoho API to fetch active credit trips
ENROUTE_TRIPS_URL	Zoho API to fetch active enroute/Live Routes trips
REPORT_TO	Number that receives the daily WhatsApp report (set in Render environment variables)
TOKEN_SECRET	Secret key used to generate secure truck tracking link tokens
DASHBOARD_KEY	Password to access the dashboard (ml2026secure)

4. Database Tables

The database is hosted on Neon (cloud PostgreSQL). There are 5 tables. You connect to it using the DATABASE_URL environment variable.

4.1 trucks

All trucks connected to the FASTag system.

Column	Type	What it stores
vehicle_no	TEXT	Truck number e.g. TN12AU2754
owner	TEXT	Owner / company name
state	TEXT	State code — TN, KA, HR etc
is_connected	INTEGER	1 = FASTag connected, 0 = not connected
fetched_at	TEXT	Last time FASTag data was fetched (stored as IST)

4.2 crossings

Every FASTag toll crossing ever recorded. This is the largest table.

Column	Type	What it stores
vehicle_no	TEXT	Truck number
plaza	TEXT	Toll plaza name
crossed_at	TIMESTAMP	Date and time of crossing — stored in IST
lat	FLOAT	Latitude of the toll plaza
lng	FLOAT	Longitude of the toll plaza

4.3 enroute_trips

Active long-distance trips from Zoho. Deleted and re-inserted on every sync (11 AM and 7 PM).

Column	Type	What it stores
truck_no	TEXT	Truck number
tranco	TEXT	Transport company name
shipper	TEXT	Shipper — Godrej, Marico, KSK etc

Column	Type	What it stores
loading_point	TEXT	Loading location name e.g. Tiruporur
reporting_date	TEXT	Date truck was loaded (YYYY-MM-DD)
reporting_time	TEXT	Time truck was loaded (HH:MM:SS)
pickup_pin	TEXT	Origin pincode
drop_pin	TEXT	Destination pincode
trip_status	TEXT	"active" or "completed"
near_dest_since	TIMESTAMP	When truck first came within 50 km of destination

4.4 credit_trips

Active short-distance trips. Fully replaced on every sync — old data deleted, fresh data inserted.

Column	Type	What it stores
truck_no	TEXT	Truck number
pickup	TEXT	Pickup pincode
drop_pin	TEXT	Destination pincode
tranco	TEXT	Transport company
lr_number	TEXT	LR (Lorry Receipt) number
lr_date	TEXT	Date of the LR
synced_at	TIMESTAMP	When this was synced from Zoho

4.5 pincode_cache

Caches pincode → coordinates lookups from Nominatim (OpenStreetMap). Avoids repeated geocoding calls.

Column	Type	What it stores
pincode	TEXT	Indian pincode — primary key
lat	FLOAT	Latitude for this pincode
lng	FLOAT	Longitude for this pincode
city	TEXT	City name for this pincode

5. Schedule — What Runs When

APScheduler runs inside wsgi.py and fires jobs at fixed times every day in IST. A file lock at /tmp/hundred_scheduler.lock ensures only one server process starts the scheduler — even if Gunicorn has multiple workers.

Night-time ping pause (12 AM – 6 AM)

FASTag pings for enroute trucks originally ran every hour, 24 hours a day (24 pings/day). Since the dashboard is only monitored during office hours and no one tracks trucks at night, pings are now skipped from 12 AM to 5 AM. The 6 AM ping catches all overnight toll crossings in one go — no data is lost, just slightly delayed until morning. This reduces pings from 24/day to 18/day, saving ~25% of Neon PostgreSQL compute usage.

UptimeRobot — keeping Render awake 24/7

Render free-tier services sleep after 15 minutes of inactivity. UptimeRobot is configured to ping the app every 5 minutes, which prevents it from ever sleeping — including during the 12 AM–6 AM window when FASTag pings are paused. This means the APScheduler keeps running overnight and the 6 AM ping fires on time. During the no-ping window, UptimeRobot keeps the server alive but the database is not queried, so Neon compute usage is still saved. The two work together: UptimeRobot = server always up, night pause = database usage reduced.

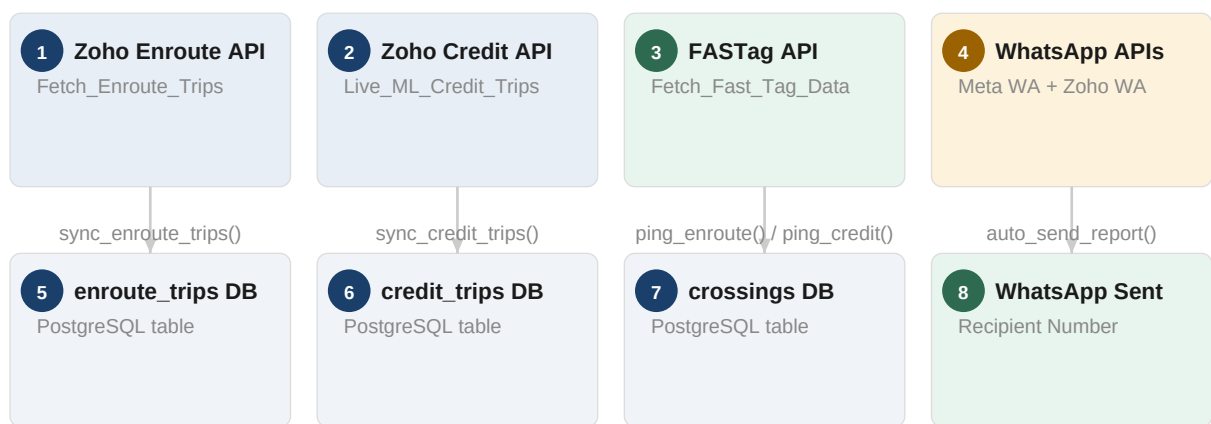
TIME (IST)	FUNCTION	WHAT IT DOES
6 AM – 11 PM :00	ping_enroute	Fetch FASTag crossings for all enroute trucks (18×/day)
11:00 AM	sync_enroute_trips	Fetch fresh trip list from Zoho Enroute API
7:00 PM	sync_enroute_trips	Evening Zoho sync for enroute trips
11:05 AM	auto_send_report	Send WhatsApp report + tracking links
7:05 PM	auto_send_report	Send WhatsApp report + tracking links
9:00 AM	sync_credit_trips ping_credit	Fetch credit trips from Zoho + FASTag ping
1:00 PM	sync_credit_trips ping_credit	Zoho sync + FASTag ping for credit trucks
5:00 PM	sync_credit_trips ping_credit	Evening Zoho sync + FASTag ping for credit

How to change a schedule time

Open wsgi.py and search for "APScheduler started" near the bottom of the file. Each job line looks like: `scheduler.add_job(ping_enroute, "cron", minute=0)`. Change the `hour=` and `minute=` values. Push wsgi.py to Render. The new schedule takes effect after redeploy.

5.1 Data Flow Diagram

This shows how each scheduled job connects a source to a destination.



6. API Endpoints

All endpoints are served by wsgi.py. Base URL:
<https://hundred-trucks-analysis.onrender.com>

Endpoint	What it returns / does
/api/hundred-trucks	List of all trucks with crossing count and owner
/api/hundred-truck-detail	All crossings for a specific truck (pass ?vehicle_no=XX)
/api/hundred-all-crossings	All crossings for all trucks — used by the map view
/api/hundred-routes	Route lines for All Routes map view
/api/enroute-trips	Active enroute trips with FASTag crossing data — powers Live Routes
/api/credit-trips	Active credit trips with destination coordinates
/api/credit-truck-route	FASTag crossings for a specific credit trip truck
/api/track	Truck data for the public tracking page (uses secure token)
/api/make-token	Generate a tracking link token for a truck number
/api/send-report	Manually trigger WhatsApp report immediately
/api/force-sync	Force sync both Zoho APIs right now — use when map is stuck or empty
/api/add-enroute	Manually add a truck to the enroute trips list
/api/remove-enroute	Remove a truck from the enroute trips list
/api/ping-truck	Manually trigger FASTag data fetch for one specific truck

7. Dashboard Views

The dashboard is a single HTML file — `hundred_trucks.html`. It has 6 views accessible from the top navigation bar. All data is fetched from the backend API on the fly.

Trucks

Default view

Lists all 100 trucks with crossing count. Filter by state or Connected. Click a truck to see all toll crossings in a table, timeline, and journey map.

Map View

All trucks on map

Shows all trucks as markers on India map. Use the time slider to replay truck positions at different times in the past.

All Routes

Route lines map

Draws route lines for all trucks simultaneously on the map. Color-coded by truck number.

Report

Plaza analysis

Toll plaza report — shows which trucks passed through each plaza. Useful for route pattern analysis.

Live Routes

Active long trips

Shows only active enroute trips. Cards show loading point, start time, last toll, and crossing count. Map shows only current position diamonds by default. Click a card to see full route.

Credit Trips

Active local trips

Shows active short-distance trips on a map. Each trip shows origin → destination. Auto-refreshes at 9 AM, 1 PM, and 5 PM.

7.1 Live Routes — detailed feature list

- Each truck card shows: loading point + pickup time as a yellow badge
- Starting time = first toll crossing that happened AFTER the pickup time (real departure)
- If no crossing found after pickup → shows scheduled loading time as fallback
- Trucks with no ping for 4+ hours show a red flag icon next to their name
- Toll Plazas button → shows all crossings as numbered circle pins on the map
- Default map = only current position diamonds. Click sidebar card = show full route
- Duplicate trucks from Zoho are deduplicated — only the latest trip per truck is kept

8. WhatsApp Reporting

WhatsApp messages are sent using the Zoho WA API (Send_WA_for_Truck_Tracking). Each message contains a truck number and its individual tracking link.

Every day at 11:05 AM and 7:05 PM IST, the server sends automatically:

```
11:00 AM sync_enroute_trips() - get latest trip list from Zoho
11:05 AM auto_send_report()   - after 11 AM Zoho sync
7:00 PM  sync_enroute_trips() - evening Zoho sync
7:05 PM  auto_send_report()   - after 7 PM Zoho sync
```

Message per live truck (Zoho WA API):
Template with truck number + individual tracking link

If WhatsApp messages stop sending

The Zoho WA API (Send_WA_for_Truck_Tracking) is used to send tracking links. If messages stop, check that the Zoho public key in wsgi.py (send_zoho_wa function) is still valid. You can test it manually by opening /api/send-report in the browser. No token renewal needed — Zoho WA uses a public key embedded in the URL.

8.1 To change the recipient number

Open wsgi.py. Find the line:

```
REPORT_TO = '<Phone number with country code e.g. +919XXXXXXX>'
```

Change to the new number. Push wsgi.py to Render.

8.2 To add more recipients

In the auto_send_report() function, duplicate the send_whatsapp() and send_zoho_wa() calls with the additional number. Each number will receive the full report independently.

9. Public Tracking Link

Each truck has a unique public link that anyone can open on their phone to see the truck's current location. The link is secure — it uses a token generated from the truck number and a secret key.

Link format:

```
https://hundred-trucks-analysis.onrender.com/track.html?token=XXXXX
```

To generate a link for any truck:

```
https://hundred-trucks-analysis.onrender.com/api/make-token?vehicle_no=TN12AU2754
```

- The token is generated from truck number + TOKEN_SECRET using HMAC-SHA256 — tamper-proof
- The page shows: truck number, last toll plaza, crossing time, and location on a map
- Auto-refreshes every 2 hours
- **Expiry date:** track.html has a constant EXPIRY = new Date('2026-08-05') around line 220
- After the expiry date, the page shows "Link Expired" — update the date and push track.html to Render

Trip Auto-Close Logic (Enroute trips)

The system has logic to automatically close an enroute trip when the truck reaches its destination. Here is how it works: when a truck's last FASTag crossing is within 50 km of the destination pincode, a timer starts. If the truck stays within 50 km for 1+ hour, the trip is automatically marked as Completed and the truck is removed from Live Routes.

This logic is built but not yet working — destination pincode missing from Zoho

For the auto-close to work, Zoho must send a drop_pincode (destination pincode) for each enroute trip. Currently Zoho returns this field as empty for all trucks — so the system has no destination to compare against and the 50 km check never runs. This is a data issue in Zoho CRM, not a code problem. Once drop_pincode is filled in Zoho for each trip, the auto-close will activate automatically.

10. How to Make Common Changes

Change schedule timings

Open wsgi.py → search "APScheduler started" → change hour= and minute= on the relevant job → push wsgi.py to Render.

Change WhatsApp recipient

Open wsgi.py → find REPORT_TO = " → change the number (include country code e.g. +91...) → push wsgi.py to Render.

Extend tracking link expiry

Open track.html → find line ~220: const EXPIRY = new Date('2026-08-05') → update to the expected end date of the current trip batch → push track.html to Render.

Fix a stuck / empty map

Open <https://hundred-trucks-analysis.onrender.com/api/force-sync> in any browser. Wait 10–15 seconds. Reload the dashboard. This forces an immediate Zoho sync.

Change database connection

Go to Render dashboard → your service → Environment → update DATABASE_URL → redeploy. No code change needed.

Push code to production

Edit files locally → commit to Git → push to main branch → Render auto-deploys. After deploy the scheduler restarts and runs a startup sync automatically.

WhatsApp messages stopped sending

The app uses Zoho WA API (Send_WA_for_Truck_Tracking). Test by opening /api/send-report in the browser. If it works manually but not on schedule, check Render logs for errors at 11:05 AM / 7:05 PM. No token renewal needed — Zoho WA uses a public key in the URL.

11. Known Issues and Things to Watch

Duplicate WhatsApp messages

If Gunicorn runs more than 1 worker, the scheduler fires in every worker and sends duplicate messages. The file lock at `/tmp/hundred_scheduler.lock` prevents this. Also ensure the Render start command includes `--workers 1`.

Neon PostgreSQL compute limit (free tier)

Neon free tier has a monthly compute allowance. FASTag pings were originally 24/day; reduced to 18/day by skipping 12 AM–5 AM (no one monitors at night). If compute approaches 100%, either wait for the monthly reset or upgrade the Neon plan. To further reduce usage, widen the night pause window in `wsgi.py`: change `hour='6-23'` to e.g. `hour='7-22'`.

IST Timezone

All timestamps are stored and shown in IST. The server calculates IST as UTC + 5h30m explicitly — never using the server's local timezone (which is UTC on Render). The frontend always appends +05:30 when parsing timestamps. If timestamps look wrong, check both sides.

Trucks with no FASTag data

Some trucks appear in Zoho but have 0 FASTag crossings — the FASTag may be inactive or disconnected. Currently: TN72CL6728 (0 crossings since 3 July), TN49A9809 (new truck). Follow up with the transporter to activate or register the FASTag.

Trip completion logic built but not triggering — missing drop_pincode from Zoho

The logic is built: when a truck stays within 50 km of its destination for 1+ hour, the trip is marked completed automatically. However, Zoho's `Fetch_Enroute_Trips` API returns empty `drop_pincode` for all trucks — so the distance calculation is never possible and the logic never triggers. This is a data issue in Zoho, not a code issue. To fix: ensure `drop_pincode` is filled in Zoho CRM for each enroute trip, or check if the pincode is stored under a different field name in the API response.

Pincode geocoding accuracy

Origin/destination coordinates come from Nominatim (OpenStreetMap). Some pincodes land in wrong places. Results are cached in the `pincode_cache` table. To fix a wrong location: delete that pincode row from `pincode_cache` — it will be re-fetched and re-cached on next load.

12. Environment Variables on Render

Go to Render → your service → Environment to set these. Never hardcode secrets in the code itself. On your local machine they go in the .env file.

Variable	What it is	Where to get it
DATABASE_URL	PostgreSQL connection string	Neon dashboard → Connection string
WA_TO	Default WhatsApp recipient number	Set manually (include country code, e.g. +91...)
TOKEN_SECRET	Secret for tracking link tokens	Set any long random string — keep it stable
DASHBOARD_KEY	Dashboard login password	Set any strong string — currently ml2026secure

Render Start Command

```
gunicorn wsgi:application --worker-class sync --workers 1 --preload --timeout 120
```

--workers 1 is critical

Multiple Gunicorn workers cause every scheduled job to run once per worker — resulting in duplicate WhatsApp messages. Always keep workers at 1 for this project.

13. Zoho API Reference

API Name	What it provides	Used in
Fetch_Enroute_Trips	Active long-distance trips — truck number, loading point, pickup date/time, origin pincode, drop pincode, shipper, transporter	sync_enroute_trips() 11 AM + 7 PM
Live_ML_Credit_Trips	Active credit (short-distance) trips — truck number, pickup pincode, drop pincode, LR number, LR date, transporter	sync_credit_trips() 9 AM + 5 PM
Fetch_Fast_Tag_Data	FASTag toll crossing data for one truck — plaza name, crossing time, latitude, longitude	ping_enroute() hourly ping_credit() 3x/day
Send_WA_for_Truck_Tracking	Sends a WhatsApp template message with tracking link to a given phone number via Zoho WA	auto_send_report() — 11:05 AM + 7:05 PM

14. Quick Reference

Important URLs

Dashboard (requires password key):

<https://hundred-trucks-analysis.onrender.com/?key=ml2026secure>

Force sync (when map is empty):

<https://hundred-trucks-analysis.onrender.com/api/force-sync>

Manually trigger WhatsApp report:

<https://hundred-trucks-analysis.onrender.com/api/send-report>

Generate tracking link for a truck:

https://hundred-trucks-analysis.onrender.com/api/make-token?vehicle_no=TN12AU2754

Problem → Solution

Problem	Solution
Map is empty, no trucks showing	Open <code>/api/force-sync</code> URL, wait 15 seconds, reload dashboard
Timestamps showing UTC not IST	Check <code>fetch_at</code> in <code>wsgi.py</code> is using IST calculation, not <code>time.strftime()</code>
WhatsApp messages not sending	Test manually via <code>/api/send-report</code> . If it works, check Render logs at 11:05 AM / 7:05 PM for scheduler errors. Zoho WA needs no token renewal.
Duplicate WhatsApp messages	Set <code>--workers 1</code> in Render start command and redeploy
Truck not in Live Routes	Check if truck is in Zoho <code>Fetch_Enroute_Trips</code> API; add manually via <code>/api/add-enroute</code>
Credit Trips map stuck loading	Open <code>/api/force-sync</code> to immediately resync credit trips from Zoho
Tracking link shows "Link Expired"	Update <code>EXPIRY</code> date in <code>track.html</code> line ~220 and push to Render
Wrong location on map for a city	Delete that pincode row from <code>pincode_cache</code> table — it will be re-geocoded
Scheduler not running after deploy	Check Render logs for "APScheduler started" message after deploy
Trip never marks as completed	FASTag data must be coming for that truck — check <code>crossings</code> table for recent data

